ROB 498/599 · 3D Robot Perception · Fall 2026

L16 Latent dynamics

World models that predict what happens next under the robot's own actions, with the probability, recurrent networks, VAEs and reinforcement learning they rest on: planning with CEM, PlaNet, Dreamer and DreamerV3, and JEPA's prediction in representation space.

Mon Nov 2, 2026 • WRITE-IN build

Latent dynamics

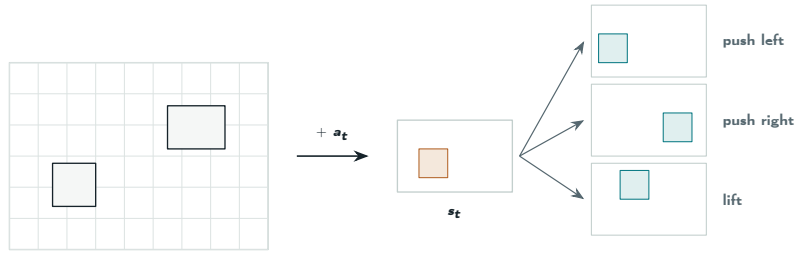
Every map so far has described the world as it is. A robot acts, so it also needs to know what the world will look like after it acts.

Unit 3, **world models and prediction**: from a static model of what is there to a predictive model of what happens next under the robot's own actions.

Today

1. **World models**, and the probability they are written in.
2. **Compounding error** and **planning** with a model: MPC, and CEM by hand.
3. **Learning a latent state:** likelihood, KL, autoencoders, the VAE.
4. **Recurrent networks**, then **PlaNet**'s recurrent state-space model.
5. *Continued in L17:* PlaNet, reinforcement learning, Dreamer and DreamerV3, and I-JEPA.

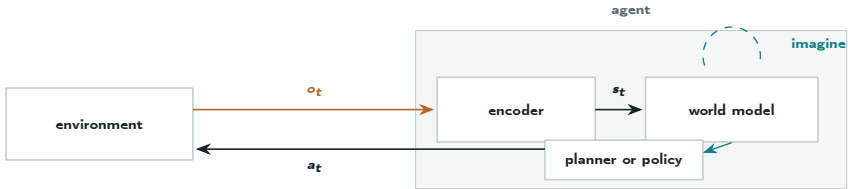
From what is there to what happens next



orange: the object as observed now; teal: where the model predicts it goes

- Units 1 and 2 built maps of the present: occupancy, TSDFs, Gaussians, scene graphs.
- A **world model** predicts the next state given the current one **and an action**.

Why a robot wants one

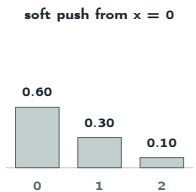


orange: the real observation; teal: the model's imagined futures

- **Plan:** try action sequences inside the model, run the best one.
- **Imagine:** train a policy on rollouts the model makes up, with no robot time.
- **Fewer real trials:** PlaNet matches a model-free agent's scores with 1,000 episodes instead of 100,000.
- **Safety:** a bad action tried in the model breaks nothing.

Probability

1 of 6: a random variable; illustrative numbers



Push a block on a table. It slides 0, 1 or 2 cm, never the same twice.

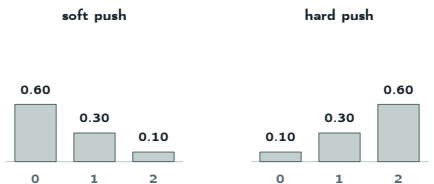
A **random variable** X is a quantity whose value is uncertain. Its **distribution** gives each value a probability:

$$p(X{=}0) = 0.6, \quad p(X{=}1) = 0.3, \quad p(X{=}2) = 0.1$$

Probabilities are ≥ 0 and sum to 1.

Probability

2 of 6: conditional distributions



$$p(x' \mid x, a): \quad p(x'{=}2 \mid x{=}0, \text{soft}) = 0.1, \qquad p(x'{=}2 \mid x{=}0, \text{hard}) = 0.6$$

A **conditional distribution** is one distribution per value of what we condition on. Each one sums to 1. A world model is exactly this: $p(s' \mid s, a)$.

Probability

3 of 6: expectation

$$\mathbb{E}[X] = \sum_x x\, p(x) \qquad \text{or, for continuous } x, \qquad \mathbb{E}[X] = \int x\, p(x)\, dx$$

Expected slide for the soft push, and for the hard push. Then average these 8 observed soft pushes: 0, 1, 0, 0, 2, 0, 1, 0.

The **expectation** is the probability-weighted average. With samples $x_1, \ldots, x_N$ from p, $\frac{1}{N}\sum_i x_i \approx \mathbb{E}[X]$.

Probability

4 of 6: variance and standard deviation

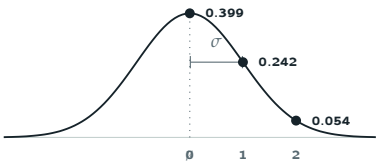
$$\mathrm{Var}[X] = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big], \qquad \sigma = \sqrt{\mathrm{Var}[X]}$$

Variance and standard deviation of the soft push (mean 0.5).

The mean says where; the standard deviation says how spread out. A model that predicts both says how sure it is.

Probability

5 of 6: the Gaussian



$$\mathcal{N}(x;\ \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\Big(-\frac{(x-\mu)^2}{2\sigma^2} \Big)$$

- A distribution over a continuous x: a **density**, whose area is 1.
- Two numbers fix it: the mean μ and the standard deviation σ.

Evaluate $\mathcal{N}(x; 0, 1)$ at $x = 0,\ 1,\ 2$.

Probability

6 of 6: a Gaussian in many dimensions

With d independent dimensions (a **diagonal** Gaussian), the density is the product of d one-dimensional ones, so the log density is a sum:

$$\ln \mathcal{N}(\mathrm{z};\ \boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2)) = \sum_{i=1}^{d} \ln \mathcal{N}(z_i;\ \mu_i, \sigma_i^2)$$

- $\ln \mathcal{N}(\mathrm{z}; 0, I)$ **for** $\mathrm{z} = (0.5, -1)$.

PlaNet's stochastic state is a 30-dimensional diagonal Gaussian.

A world model, written down

If the state is known (joint angles, object poses), the model is a conditional distribution:

$$s_{t+1} \sim p(s_{t+1} \mid s_t,\ a_t)$$

From cameras the state is not observed, so predict the next frame from everything seen and done:

$$o_{t+1} \sim p(o_{t+1} \mid o_{1:t},\ a_{1:t})$$

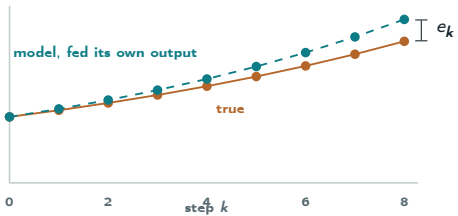
Latent models split this into an **encoder** $o_t \mapsto s_t$, a **transition** $p(s_{t+1} \mid s_t, a_t)$, and a **decoder** $s_t \mapsto o_t$.

What a world model is conditioned on

Conditioned on	What it can answer
past frames only	what usually happens next (video prediction). No use for choosing an action.
past frames **and actions**	what happens if *I* do this: planning, imagination training. Today's models.
a goal image or text	which actions reach the goal: the planner compares predictions with it.
a policy	what happens if I keep behaving like this: the value of a policy, without running it.

A one-step model, rolled out

illustrative numbers

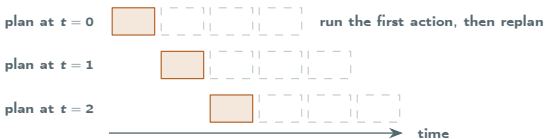


teal: produced by the model; orange: observed or true

- Trained on one step: given the true s_t, predict s_{t+1}.
- Used for k steps: the model's output becomes its next input.
- True system $x_{k+1} = 1.1\,x_k$, model 1.12. After 8 steps: off by 0.33.

Planning as search over action sequences

$$a^{\star}_{t:t+H-1} = \arg\min_{a_{t:t+H-1}} \sum_{k=0}^{H-1} c(\hat{s}_{t+k},\ a_{t+k}), \qquad \hat{s}_{t+k+1} = f(\hat{s}_{t+k}, a_{t+k}),\quad \hat{s}_t = s_t$$



dashed: planned in the model; orange: run on the robot

- **Model-predictive control (MPC):** solve for H actions, run only the first, observe, solve again.
- The cost c can be a predicted reward or a distance to a goal.
- The model need not be differentiable: the next methods only *evaluate* it.

Random shooting, then CEM

Random shooting: sample J sequences, score each in the model, run the first action of the best. Fine for small searches; PlaNet's cheetah has 6 action dimensions over 12 steps, a 72-dimensional search.

Cross-entropy method (CEM): sample near the best ones and repeat.

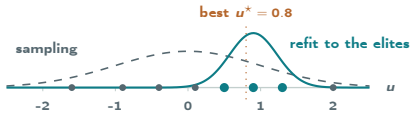
1. Gaussian over action sequences, $\mu = 0$, $\sigma = 1$.
2. Sample J sequences; score each in the model.
3. Keep the best K, the **elites**.
4. Refit μ, σ to the elites. Repeat I times.
5. Run the first action of μ; next step, start again from $\mathcal{N}(0, 1)$.

PlaNet: $H = 12$, $I = 10$, $J = 1{,}000$, $K = 100$: 120,000 predictions per action.

CEM by hand

1 of 2: one round, illustrative numbers

One action u, robot at $x = 0$, goal at 1, model $x' = x + u$. Cost $J(u) = (u-1)^2 + 0.25\,u^2$: reach the goal without pushing hard.

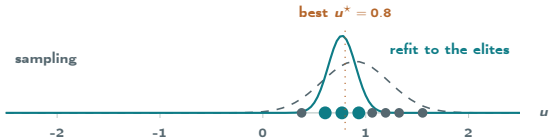


teal: the elites and the refitted Gaussian; orange: the true optimum

Eight draws from $\mathcal{N}(0,1)$: $-1.6,\ -0.9,\ -0.4,\ 0.1,\ 0.5,\ 0.9,\ 1.3,\ 2$. Score each, keep the best 3, refit μ and σ.

CEM by hand

2 of 2: the second round



teal: the elites and the refitted Gaussian; orange: the true optimum

Same eight z, now $u = 0.90 + 0.327\,z$. Which three win, and where does μ go?

Why plan in a latent space?

- PlaNet sees 64×64 RGB images: 12,288 numbers per frame. Its latent state is $200 + 30 = 230$ numbers, 53 times fewer.
- CEM needs 120,000 predictions per action: out of reach for images, affordable for small vectors.
- Pixels also carry what does not matter for the task: texture, lighting, the background. A latent can drop it.

The tool for learning a small state from images is the **variational autoencoder (VAE)**. It is trained by maximum likelihood with a KL penalty, so those come first.

Likelihood

illustrative numbers

Three measurements of the block's position: 0.8, 1.1, 1.4. Model: each is drawn from $\mathcal{N}(\mu, 1)$. Which μ explains them better?

$$L(\mu) = \prod_i \mathcal{N}(x_i;\ \mu, 1), \qquad \ln L(\mu) = -\tfrac{1}{2}\sum_i (x_i - \mu)^2 - \tfrac{3}{2}\ln 2\pi$$

$\ln L$ for $\mu = 1.1$ and $\mu = 0.5$.

The **likelihood** is the probability (density) of the observed data, read as a function of the model's parameters.

Maximum likelihood, and why MSE is one

Maximum likelihood: pick the parameters that make the data most likely. Here, set $\frac{d}{d\mu} \ln L = \sum_i (x_i - \mu) = 0$: $\mu =$ the mean $= 1.1$.

For one measurement x and a network's prediction $\hat{x}$, a unit-variance Gaussian gives

$$-\ln \mathcal{N}(x;\ \hat{x}, 1) = \tfrac{1}{2}(x - \hat{x})^2 + \tfrac{1}{2} \ln 2\pi$$

Evaluate it for $x = 3$, $\hat{x} = 2.5$.

So minimising mean squared error (L05) **is** maximum likelihood under a Gaussian. Cross-entropy (L05) is the same for a softmax.

Entropy

illustrative numbers

$$H(p) = -\sum_x p(x) \ln p(x)$$

Entropy of a fair coin $(0.5, 0.5)$**, and of a biased coin** $(0.9, 0.1)$**.**

Entropy measures how uncertain a distribution is: the average surprise $-\ln p(x)$ of its own samples. Zero when one outcome is certain.

From cross-entropy to KL divergence

illustrative numbers

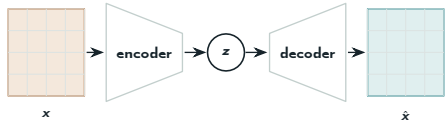
$$H(p, q) = -\sum_x p(x) \ln q(x) \quad \text{(L05)}, \qquad \mathrm{KL}(p \,\|\, q) = H(p, q) - H(p) = \sum_x p(x) \ln \frac{p(x)}{q(x)}$$

p = **fair coin,** $q = (0.9, 0.1)$**. Find** $H(p, q)$**, then** $\mathrm{KL}(p \,\|\, q)$ **and** $\mathrm{KL}(q \,\|\, p)$**.**

KL divergence: the extra surprise from using q when the data come from p. Always ≥ 0, and 0 only when $q = p$.

Autoencoders

1 of 2: the idea



orange: the input image; teal: the model's reconstruction

- An **encoder** maps an image x to a short code z; a **decoder** maps z back to $\hat{x}$.
- Train both on $\|x - \hat{x}\|^2$ by gradient descent (L05). The bottleneck forces z to keep what is needed to redraw x.
- **Self-supervised:** the target is the input itself, so no labels are needed.

Autoencoders

2 of 2: by hand, illustrative numbers

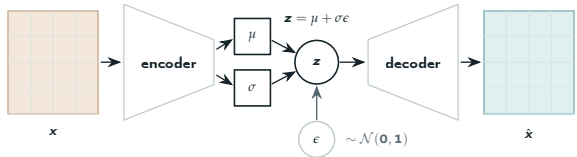
A linear autoencoder from 2 numbers to 1 and back. Encoder $z = 0.8\,x_1 + 0.6\,x_2$; decoder $\hat{x} = (0.8\,z,\ 0.6\,z)$.

Encode $x = (2, 1)$, decode, and find the squared error.

One number cannot hold two exactly; training moves the weights to lose as little as possible. Nothing says what a code *between* training codes decodes to.

Variational autoencoders

1 of 6: a distribution per input



orange: the input image; teal: the model's reconstruction

- The encoder outputs a mean $\mu(x)$ and a standard deviation $\sigma(x)$: a Gaussian $q(z \mid x)$ per input.
- Every code near μ must decode to something like x.
- **Analogy:** an autoencoder pins each image to one point on a map. A VAE gives each image a small district, so nearby points still decode to something.

Variational autoencoders

2 of 6: the reparameterisation, by hand

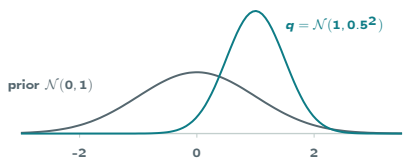
$$z = \mu + \sigma\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0, 1)$$

The encoder gives $\mu = 1$, $\sigma = 0.5$; the noise draw is $\epsilon = -0.4$. Find z, $\partial z/\partial \mu$ and $\partial z/\partial \sigma$.

Sampling becomes a sum and a product, so backpropagation (L05) passes through it to the encoder.

Variational autoencoders

3 of 6: keep the clouds together



teal: the encoder's output for one image; grey: the shared prior

The encoder could cheat: shrink $\sigma \to 0$ and spread the means apart. So penalise the KL from each $q(z \mid x)$ to a shared **prior** $p(z) = \mathcal{N}(0, 1)$:

$$\mathrm{KL}\big(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0,1)\big) = \tfrac{1}{2}\left(\mu^2 + \sigma^2 - 1 - \ln \sigma^2\right)$$

Variational autoencoders

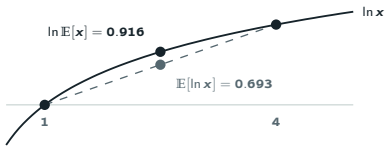
4 of 6: the KL by hand

$$\mathrm{KL} = \tfrac{1}{2}\left(\mu^2 + \sigma^2 - 1 - \ln \sigma^2\right)$$

Evaluate it for $\mu = 1,\ \sigma = 0.5$; for $\mu = 0,\ \sigma = 1$; and for the cheat $\mu = 0,\ \sigma = 0.1$. What does each term punish?

Variational autoencoders

5 of 6: Jensen's inequality



$$\ln \mathbb{E}[x] \;\geq\; \mathbb{E}[\ln x]$$

- $x = 1$ **or** 4**, equally likely.**

ln curves downward, so the log of an average is above the average of the logs.

Variational autoencoders

6 of 6: the ELBO

The model: draw a code $z \sim p(z)$, decode it with $p(x \mid z)$. Maximum likelihood wants $\ln p(x) = \ln \int p(x \mid z)\, p(z)\, dz$, but that integral is intractable.

- **Multiply and divide inside the integral by $q(z \mid x)$, then use Jensen's inequality.**

The **evidence lower bound (ELBO)**. Maximise it: decode well, and keep the codes near the prior.

The ELBO, by hand

one sample, illustrative numbers

One input $x = 1$. The encoder gives $q = \mathcal{N}(1, 0.5^2)$; the noise draw $\epsilon = -0.4$ gives $z = 0.8$ (two slides back). The decoder predicts $\hat{x} = z$, unit variance.

Estimate the ELBO from this one sample: reconstruction term, minus the KL.

Training averages this over a batch and follows its gradient. The expectation is estimated with one sample.

Recurrent networks

1 of 2: a network with memory

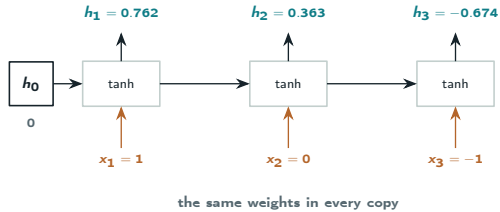
$$h_t = \tanh\left(Wh_{t-1} + Ux_t + b\right)$$

- An MLP (L05) sees one input. A **recurrent neural network (RNN)** reads a sequence one step at a time.
- The **hidden state** h_t is its memory: each step mixes the old memory with the new input.
- The same weights W, U, b are used at every step.

A world model needs this: the next frame depends on the whole history of frames and actions.

Recurrent networks

2 of 2: unrolled, by hand, illustrative numbers



orange: the input sequence; teal: the hidden states the network computes

Scalar RNN, $W = 0.5$**,** $U = 1$**,** $b = 0$**,** $h_0 = 0$**. Run it on** $x = 1, 0, -1$**.**

Gated memory

1 of 3: keep or overwrite, illustrative numbers

A plain RNN replaces its memory at every step. A **gated** network blends: the new memory is a point between the old memory and a **candidate**.

$$h_t = (1 - z)\, h_{t-1} + z\, \tilde{h}$$

z, the **update gate**, is between 0 and 1.



teal: the new memory at z = 0.2

Old memory $h_{t-1} = 0.8$. Candidate $\tilde{h} = \tanh(0.5 \cdot 0.8 - 1) = -0.537$, the plain RNN's answer. Blend with $z = 0,\ 0.2,\ 1$.

Gated memory

2 of 3: the network sets the gate, illustrative numbers

$$z = \sigma(w_z\, h_{t-1} + u_z\, x_t)$$

Made-up weights $w_z = 1$, $u_z = 2$; $h_{t-1} = 0.8$. Find z and h_t for $x_t = -1$, then for $x_t = +1$.

One small neuron, with a sigmoid (L05) so $0 \le z \le 1$. Training learns w_z, u_z: the network learns **when** to remember.

Gated memory

3 of 3: the reset gate, and the whole GRU

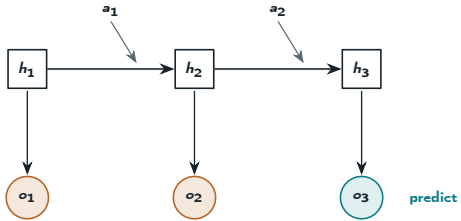
$$r = \sigma(w_r h_{t-1} + u_r x_t) \quad \text{reset gate}$$
$$z = \sigma(w_z h_{t-1} + u_z x_t) \quad \text{update gate}$$
$$\tilde{h} = \tanh\left(W\left(r\, h_{t-1}\right) + U x_t\right) \quad \text{candidate}$$
$$h_t = (1 - z)\, h_{t-1} + z\, \tilde{h} \quad \text{blend}$$

New piece: r sets how much of the old memory the candidate sees. r near 0 is a fresh start.

Same $h_{t-1} = 0.8$, $x_t = -1$, $z = 0.231$ from the last slide. Made-up $w_r = u_r = 1$.

A latent dynamics model

1 of 4: deterministic only

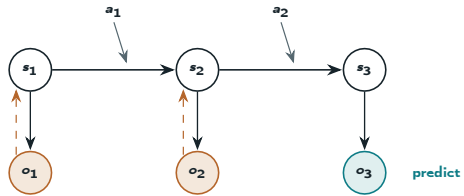


orange: observed frames; teal: the predicted frame

- A recurrent network: $h_t = f(h_{t-1}, a_{t-1})$, each h_t decoded to a frame.
- Remembers well, but predicts **one** future: it cannot say *the door might open or not*.

A latent dynamics model

2 of 4: stochastic only

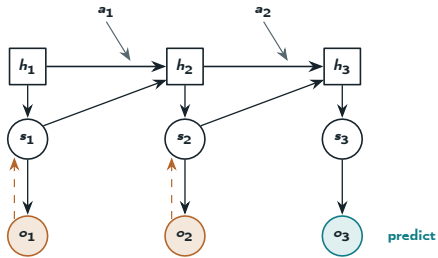


orange: observed frames and what is inferred from them; teal: predicted

- A state-space model: $s_t \sim p(s_t \mid s_{t-1}, a_{t-1})$, a Gaussian per step, so several futures can be represented.
- Dashed: an encoder infers s_t from the frame it saw, a VAE per step.
- All memory passes through one noisy sample per step, so long-range information is lost.

A latent dynamics model

3 of 4: both paths (RSSM)



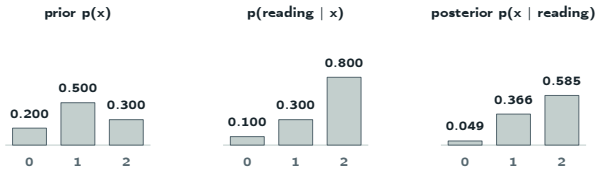
orange: observed frames and what is inferred from them; teal: predicted

$$h_t = f(h_{t-1},\ s_{t-1},\ a_{t-1}), \qquad s_t \sim p(s_t \mid h_t), \qquad o_t \sim p(o_t \mid h_t, s_t), \quad r_t \sim p(r_t \mid h_t, s_t)$$

The **recurrent state-space model**: h_t (a GRU, 200 units) carries memory; s_t (30-D diagonal Gaussian) carries uncertainty.

Bayes' rule: prior to posterior

illustrative numbers



$$p(x \mid o) = \frac{p(o \mid x)\, p(x)}{p(o)}, \qquad p(o) = \sum_{x} p(o \mid x)\, p(x)$$

Before looking, the block is at $0, 1, 2$ with $0.2, 0.5, 0.3$. A sensor says "near 2", which it does with probability $0.1, 0.3, 0.8$ from each position. Where is the block now?

Learn it again, differently

other explanations

- Lilian Weng, From Autoencoder to Beta-VAE: the VAE and the ELBO step by step.
- Ha and Schmidhuber, World Models (2018): a car racer trained inside its own dream.
- Hou et al., World Model for Robot Learning: A Comprehensive Survey (2026), with a project page.

What you should be able to do

- Compute an expectation, a Gaussian density, a log-likelihood, an entropy and a KL; say why MSE is a Gaussian likelihood.
- Explain why a one-step model drifts when rolled out; run two rounds of CEM by hand.
- Run a VAE forward by hand: encode, reparameterise, decode, ELBO. Derive the ELBO.
- Step an RNN and a GRU by hand; draw the RSSM's two paths and say what each carries.
- Apply Bayes' rule to a prior and a sensor reading.

Next

L17 Wed Nov 4

- **L17, Wed Nov 4:** we finish today's material (the RSSM's loss, PlaNet, reinforcement learning, Dreamer and DreamerV3, I-JEPA), then **4D and occupancy forecasting**. Response paper: Occupancy Networks.
- **Responses due Sun Nov 8:** DreamerV3 and Occupancy Networks.
- Also assigned for Unit 3: PlaNet, I-JEPA, V-JEPA, V-JEPA 2, and the survey.
- **Seminars, student-led:** Mon Nov 9 (Unit 2) and Wed Nov 11 (Unit 3). I am at CoRL that week.